

S07 Transactional Booking your individual route

FINAL_LOCAL kit v1.2.1 · local Phase4 closure · EN-GB · U07 - Relationships, Transactions and API Design

One booking changes three related facts: available seats, a Booking row and a BookingAudit row. Your task is to keep those facts consistent when a later operation fails. Complete one implementation in `02_PROJECTS/p02/src/book-seats.js`, a bounded actual Gemini review and the required P03-informed architecture decision record (ADR). Full P01 and P03 implementation is optional and is not necessary for the standard maximum mark.

The meeting has 60 minutes of content within a 90-minute slot. The other 30 minutes are logistics. Stop new work at content minute 60. The 28-minute implementation slot is time for progress, not a promise that every student finishes. Complete the remaining standard evidence before the later owner-configured deadline; this kit does not invent a date.

This guide includes expected values derived from source. They are not your observations. No canonical application, database driver or canonical test was executed during local production and Phase4 closure. Native Windows, macOS, browser storage/clipboard/print, Word, Gemini and Moodle acceptance are pending. The drawings are illustrative schematics. Record the interface and versions you actually use.

First decide which work is available

The standard final route needs genuine supplied Sequelize/native SQLite execution, actual local HTTP, the ADR and an actual bounded Gemini interaction checked independently. If a prerequisite is missing, record the exact block and continue with source reasoning or labelled synthetic practice. Do not call a missing driver, timeout or version mismatch an intended objective failure. Synthetic practice does not automatically replace standard final evidence. A form selection cannot grant an alternative: a separate teacher decision must exist with an exact scope and reference.

Open `01_WORKSHEET/index.html` for the evidence form. Keep its exported JSON draft privately. The JSON is an editable backup, not the final submission and not proof that the recorded work happened. The final submission is one readable `TW2026_S07_GROUP_Surname_Firstname.pdf` in the S07 Assignment. C07 context does not create a second Assignment.

The six content segments

Content time	Work	Save before moving on
0-6	Two prior predictions and the three-part invariant	Named fixture, requests, prediction A and prediction B
6-12	Readiness and bounded unsafe comparison	Actual readiness output or exact pending reason
12-40	Individual P02 implementation	Only the permitted file changed; current progress

Content time	Work	Save before moving on
40-49	Success, two failures, transaction trace and recovery	Before/after, error origin, five operations, same-instance recovery
49-55	One bounded actual Gemini audit	Prompt, claim, independent check, verdict, correction and limit
55-60	Required ADR handoff, save and STOP	Draft backup, exit ticket and smallest remaining check

Evidence vocabulary

Class	What it establishes
SOURCE_REASONING	You read named code or a contract and reason from it. No execution is implied.
SOURCE_EXPECTATION	A predicted or source-derived result awaiting measurement.
MODULE_MODEL	A declared stand-in model ran. Its runtime and assumptions must be named; it is not the genuine database.
SYNTHETIC_PRACTICE	A constructed teaching example or helper ran. It does not establish native application behaviour.
GENUINE_ORM_SQLITE	The supplied ORM and native SQLite stack actually ran on an owned fixture.
LOCAL_HTTP	An actual request reached the supplied local application. A helper or a file page alone does not qualify.
ACTUAL_GEMINI	The actual sanitised interaction is recorded. Its claim still needs an independent check.
NATIVE_BROWSER / NATIVE_WINDOWS / NATIVE_MACOS / NATIVE_WORD	The named native surface was actually checked. An SVG or container render does not establish this.
NOT_EXECUTED	The work did not run. Give the reason and next permitted check.

Record each witness with an ID, input/fixture, command, actual runtime, class, before/after and observed-versus-expected status. Imported content remains unverified until you review it. A structural checker can detect omissions but cannot authenticate your evidence.

Step 01 Locate the kit and distinguish the ZIP from its extracted folder

When: Before the meeting

Where: File Explorer, Finder or your usual file manager.

Exact file: The student ZIP, its .sha256 sidecar and, after extraction, 00_START_HERE/index.html.

Purpose: Work from one extracted copy and identify the kit you will use.

Action: Keep the ZIP and sidecar together. Use your normal trusted archive tool to extract once into a short local path. Windows illustration: D:\#___MY_SPACE\Downloads\s07. macOS/Linux illustration: a folder called s07 under your home Downloads folder. Spaces and # are permitted when quoted; a ZIP shown as a folder is still an archive. Open the extracted folder and double-click 00_START_HERE/index.html. The OPEN launcher, if used, only opens the guide.

ZIP versus extracted working folder

SCHEMATIC — NOT A CAPTURE OF YOUR CURRENT INTERFACE

Student archive.zip
archive.zip.sha256
Keep both original files



Extracted student folder
00_START_HERE / 02_PROJECTS
tools / PACKAGE_ID.txt

Illustrative Windows path: D:\#___MY_SPACE\Downloads\s07
Spaces and # are permitted when quoted. Open the extracted copy.
Archive preview is not the working directory.

V01 What matters: identify one real extracted copy and actual PACKAGE_ID. **What to ignore:** icons/menus differ by OS. This drawing proves neither extraction nor an integrity check.

Expected result: The address may start with file: because this guide is offline. The extracted folder contains 02_PROJECTS, 01_WORKSHEET, tools and PACKAGE_ID.txt.

Evidence: Record the archive filename, actual kit version and actual PACKAGE_ID.txt value. Retain integrity verification output when a trusted check is available; do not invent a successful hash comparison.

Likely error: You see a compressed-folder icon, missing assets or a second nested copy.

Recovery: Close the archive preview. Navigate to the real extracted folder and open its guide. Keep one working copy; do not merge files from different versions.

STOP: If the archive is corrupt, a trusted comparison differs or the expected top-level files are missing, stop application work and ask the teacher for the correct kit. Do not repair protected source by guessing.

Step 02 Open the correct folder and the one editable file

When: Before or during 0-6

Where: The extracted student folder.

Exact file: 02_PROJECTS/p02/src/book-seats.js.

Purpose: Keep project identity and the edit boundary visible.

Action: In VS Code use the usual File → Open Folder action and choose the extracted student root. In Explorer expand 02_PROJECTS, p02 and src, then select book-seats.js. Check its breadcrumb. The file currently delegates to the unsafe comparator; it is a starter, not the completed solution. Open an integrated terminal at the student root, the folder containing tools and PACKAGE_ID.txt.

One editable file and student-root terminal

SCHEMATIC — NOT A CAPTURE OF YOUR CURRENT INTERFACE

EXPLORER (illustrative)
02_PROJECTS / p02 / src
book-seats.js ← permitted edit
models.js / app.js ← preserve
tests / package files ← preserve

BREADCRUMB
02_PROJECTS > p02 > src > book-seats.js
Read contract beside your own target

TERMINAL WORKING DIRECTORY
Student root contains tools and PACKAGE_ID.txt
node tools/cli.mjs boundary p02

V02 What matters: breadcrumb and terminal root are different checks. **What to ignore:** VS Code theme and panel placement. No current editor interface was captured.

Expected result: The selected breadcrumb ends in 02_PROJECTS/p02/src/book-seats.js. Terminal commands below are issued from the student root, not from src.

Evidence: Record the target path and your working-copy identity. A sanitised crop may show the breadcrumb, but avoid personal folders or account details.

Likely error: “Cannot find module tools/cli.mjs” often means the terminal is in the wrong directory.

Recovery: Use the file manager or VS Code folder view to confirm the student root. In a Windows terminal a quoted literal path is needed for #; in a macOS/Linux shell quote the whole path. Do not paste an example path that does not exist on your machine.

STOP: If the target is absent or a reference solution has replaced the starter, stop and preserve the kit identity. Do not edit tests, dependencies, models, routes or protected comparison helpers.

Step 03 Write both predictions before observing results

When: 0-6

Where: The evidence form and the P02 contract.

Exact file: 01_WORKSHEET/index.html and contracts/P02.md.

Purpose: Make success and failure claims falsifiable without rewriting them later.

Action: Identify a fresh fixture with event 1, capacity 5, availability 5, zero bookings and zero audits. State attendee 42 and a request for 2 seats. In prediction A state the three facts you expect after a managed success. In prediction B state the three facts you expect when afterBookingCreated rejects

after the booking has been created. Record the order or time of writing. Keep the original wording even if it is wrong.

Expected result: Two separate prior predictions exist alongside their input and starting state. The source expectation is success 3/1/1, while a correct managed failure retains its before-state. These are expectations until measured.

Evidence: Preserve both predictions, the starting tuple (availableSeats, Booking count, BookingAudit count) and the intended injection point.

Likely error: You have already run a check or have only written “it works”.

Recovery: Label a prediction written after execution as retrospective. Write a new prior prediction for an unexecuted case; do not alter the old chronology. Replace vague wording with a measurable state and rejection condition.

STOP: Do not mark predictions as prior if the order is unknown. Keep the uncertainty visible.

Step 04 Measure readiness before loading the application

When: 6-9

Where: The terminal at the extracted student root.

Exact file: tools/cli.mjs and the protected project-local package metadata.

Purpose: Separate source integrity, package resolution and native qualification.

Action: Run `node tools/cli.mjs help`, then `node tools/cli.mjs verify`, `node tools/cli.mjs preflight p02` and `node tools/cli.mjs boundary p02`. Record actual output. The reference runtime is Node 24.21.0 and npm 11.19.0. P02 requires project-local Express 5.1.0, Sequelize 6.37.8 and sqlite3 6.0.1 with supplied pins/override intact. Preflight is SOURCE_RESOLUTION_ONLY_NOT_NATIVE_QUALIFICATION; a ready resolution result does not establish that the driver loads.

Expected result: You can distinguish a manifest check, an edit-boundary check and a runtime/dependency check. A permitted target edit is reported separately from protected-file drift.

Evidence: Record actual versions, resolution status, kit identity, protected-file results and any exact block. Do not copy the reference versions into the measured-version fields.

Likely error: Node is unavailable, a package is missing, a global dependency resolves, a version differs or a protected file has changed.

Recovery: Save the output. Use the labelled source-reasoning route while the teacher arranges authorised readiness later. Do not install globally, run an unrequested install, change the lockfile or suppress a guard.

STOP: Stop genuine execution until the named prerequisites and separate native-load gate are satisfied. This local closure does not authorise installation or claim that readiness has already been achieved.

Step 05 Observe the unsafe comparison only on an owned fixture

When: 9-12

Where: A ready, separately qualified environment or the source-only fallback.

Exact file: 02_PROJECTS/p02/src/unsafe-book-seats.js is read-only; the new observer controls its fixture.

Purpose: See why awaited statements alone do not make a consistency boundary.

Action: When genuine execution is authorised and ready, first use `node tools/cli.mjs native p02 --allow-native-load`, then `node tools/cli.mjs observe p02 --allow-memory-database`. The observer labels its scenarios. It compares the supplied unsafe function on a fresh owned in-memory fixture, with a rejection after booking creation. Record its actual unsafe witness. If blocked, read the comparator and use 04_OFFLINE_FALLBACK/SOURCE_REASONING_WORKSHEET.md; leave genuine execution pending.

Expected result: Source expectation for the unsafe callback failure is 3/1/0 from a fresh 5/0/0. Only an actually completed genuine witness turns this into an ORM observation. Memory fixtures do not persist across restarts.

Evidence: Name the function role, fixture, callback injection, command, actual state and evidence class. Keep an unsafe witness distinct from the managed implementation witness.

Likely error: A thrown error is recorded without the three state components or a different database is inspected.

Recovery: Repeat only when permitted, with a new named fixture and the observer's own before/after record. Do not edit the comparator or infer rows from the error text alone.

STOP: A native-driver failure, readiness block or unfinished report is not the intended unsafe counterexample. Preserve the block and stop calling it a database result.

Step 06 Read the boundary before implementing it

When: 12-15

Where: VS Code with the target and contract visible.

Exact file: 02_PROJECTS/p02/src/book-seats.js, contracts/P02.md and the read-only errors/models.

Purpose: Identify what must agree and which awaited operations belong together.

Action: Read the supplied function signature and seat validation requirement. Identify five ORM operations: `Event.findByPk`, `Booking.findOne`, `event.save`, `Booking.create` and `BookingAudit.create`. Each receives the same non-null managed transaction. The injected callback is awaited after `Booking` creation and before audit creation. Follow the source precedence: event existence, capacity, then duplicate lookup. Invalid seats are rejected before opening the transaction.

Expected result: You can explain the three-part invariant, the five database operations and the callback position without having copied a full solution.

Evidence: Write your own mechanism explanation and proposed sequence. Identify the difference between a detached return value and the outer promise settling after commit.

Likely error: You count only the four static model methods or treat “all awaits” as equivalent to a transaction.

Recovery: Include instance method `event.save`. Read the managed-transaction contract and the separate five-operation observer. The canonical four-method spy omits `save` and does not by itself establish a non-null identity.

STOP: Do not add a route, raw `BEGIN/COMMIT`, nested transaction, compensating write or process-local mutex. Do not remove tests to obtain a green result.

Step 07 Implement individually inside the one file boundary

When: 15-40

Where: Your own selected target file.

Exact file: 02_PROJECTS/p02/src/book-seats.js only.

Purpose: Implement the required consistency mechanism and keep assessment work individual.

Action: Work from the contract. Preserve the function export and supplied error types. Use one managed transaction for the reads and writes, await the injected callback and ensure the returned detached result reaches the caller after outer settlement. Validate the actual edit boundary with `node tools/cli.mjs boundary p02`. Record partial progress honestly if the slot ends before the function is complete.

Expected result: Only the permitted target differs from its distributed starter. The supplied models, tests, comparison and package files remain intact.

Evidence: Record the boundary result and explain your own change. Mark unfinished requirements rather than claiming completion from the passage of time.

Likely error: An error is replaced with a new generic error, a protected file changes or the implementation is copied without understanding.

Recovery: Undo the protected-file change using the unchanged distributed copy after identifying it. Preserve unexpected error identity according to the contract. Ask a bounded question about one mechanism rather than requesting a full assessed function.

STOP: At minute 40 save the actual state of your work. The 28-minute slot does not guarantee a correct finished implementation.

Step 08 Read success and local HTTP as separate witnesses

When: 40-43

Where: A genuinely ready environment and your current implementation.

Exact file: `tools/cli.mjs observe p02 --allow-memory-database` and the supplied P02 application.

Purpose: Distinguish rows, detached return and HTTP response.

Action: Run the guarded observer when authorised. For its managed success, record fixture identity, before/after, returned booking/event and operation trace. It also sends a local request to the supplied application. For manual use, `node tools/cli.mjs serve p02 --allow-memory-database` prints READY and the actual loopback URL; use that URL, never assume port 3000. The supplied booking route is `POST /api/events/:eventId/bookings` with the contract body. The observer is the simplest reproducible route for the required HTTP witness.

Actual local origin and supplied HTTP route

SCHEMATIC — NOT A CAPTURE OF YOUR CURRENT INTERFACE

READY prints actual `http://127.0.0.1:PORT` (port chosen by launcher)
The offline guide file: `tab` is not application HTTP evidence.

SUPPLIED POST
`/api/events/:eventId/bookings`
Expected 201 + detached data

LOCATION `/api/bookings/:id`
Names a booking identity
No supplied GET member route

HTTP status does not expose database rows. Use separate ORM witnesses.

V05 What matters: actual printed origin, method/path and distinct ORM/HTTP evidence. What to ignore: there is no live Network screenshot or guaranteed fixed port here. Do not kill foreign processes.

Expected result: The source expectation is HTTP 201 plus the detached representation after managed settlement and a two-seat state 3/1/1. HTTP 201 alone does not expose the database rows. The Location names `/api/bookings/:id`, but the supplied application has no GET booking-member route.

Evidence: Keep the actual command, printed origin, method/path/body, status, relevant response fields and ORM before/after in separate identified records. Record generated IDs as observed values.

Likely error: You open the offline guide's `file: tab`, a stale localhost tab or the Location and receive 404.

Recovery: Use the currently printed loopback origin. A Location 404 is not proof of a failed commit because no GET member route is supplied. Do not invent one. If a separate manual request is used, confirm its actual origin and avoid private data in captures.

STOP: Do not label a helper HTTP server, a static file or a source expectation `LOCAL_HTTP` for the supplied application. Do not kill a foreign process to free a preferred port.

Step 09 Keep the two failure points and recovery distinct

When: 43-46

Where: The observer's separately owned managed failure fixtures.

Exact file: The callback-failure and audit-create-failure scenarios in the observer report.

Purpose: Check rollback of prior writes and the ability to recover on the same instance.

Action: Record a callback rejection after Booking creation. Separately record a rejection in `BookingAudit.create`. For each, compare all three after-state facts with its exact before-state and record error identity/origin. Then inspect the independent successful recovery performed on that same database instance. A successful booking in a new instance is not this recovery witness.

Three state facts before and after

SCHEMATIC — NOT A CAPTURE OF YOUR CURRENT INTERFACE

SOURCE EXPECTATIONS — NOT OBSERVED PRODUCTION RESULTS

Fresh state available 5 | Booking 0 | Audit 0

Managed success available 3 | Booking 1 | Audit 1

Unsafe late failure available 3 | Booking 1 | Audit 0

Managed late failure exact before-state retained

V04 What matters: compare all three facts on named fixtures and distinguish unsafe from managed roles. **What to ignore:** these expected tuples are not your evidence. Recovery must be checked on the same failed instance.

Expected result: A correct managed implementation preserves the before-state in both failures and permits a later independent success on the same instance. These are source expectations until actually recorded.

Evidence: Use different witness IDs for callback rejection, audit rejection and each same-instance recovery. Preserve the injected error identity where the contract requires it.

Likely error: A broad error class is treated as proof of the originating constraint or a failure before insert is called an escaped audit row.

Recovery: Name the known injection origin. The reference translates `UniqueConstraintError` from the whole managed call, including possible callback or audit origins; class alone cannot identify the Booking constraint. Failure before an insert does not establish that a row escaped the transaction.

STOP: Do not claim that omitted audit transaction options necessarily produce an escaped SQLite row. A stall, native error or uncertain origin stays bounded and pending.

Step 10 Trace five operations and read finite test coverage honestly

When: 46-49

Where: The observer report and the unchanged canonical tests.

Exact file: The five-operation transaction-option trace; `02_PROJECTS/p02/tests/` remains read-only.

Purpose: Test the mechanism beyond the canonical four-method spy.

Action: For all five named ORM operations check that the transaction is non-null and identical, and preserve order/await evidence. The exact starter makes three calls counted by the canonical spy; a different incorrect control can make four calls with undefined identities. The separate observer closes that option-observation gap. When the genuine stack is ready, `node tools/cli.mjs test p02 --allow-memory-database` runs the unchanged supplied suite. Also record invalid seats, missing event, exact capacity, overcapacity and duplicate cases from the observer.

Five ORM operations inside one boundary

SCHEMATIC — NOT A CAPTURE OF YOUR CURRENT INTERFACE

ONE MANAGED TRANSACTION — same non-null option identity

1 Event.findByPk

2 Booking.findOne

3 event.save

4 Booking.create

5 BookingAudit.create

After Booking.create:
await injected callback
then audit creation

Invalid seats: before boundary. Outer result: after managed settlement.

V03 What matters: event.save is the fifth operation missed by a four-method spy. The callback is awaited between creates. **What to ignore:** box width says nothing about duration. Option tracing alone does not prove native isolation.

Expected result: Your report distinguishes option tracing, state evidence and finite suite results. Source precedence means an already reserved attendee with insufficient availability can receive sold_out before duplicate is checked.

Evidence: Record the trace provenance, suite command, exit/result and actual environment. Keep not-executed cases visible. A predicted initial objective failure is SOURCE_EXPECTATION until the genuine suite runs.

Likely error: A green four-method spy is described as universal atomicity, isolation or production durability.

Recovery: State precisely which operations and cases were covered. Sequential rollback does not qualify concurrent writers, crash durability, external-effect rollback or all possible schedules.

STOP: Do not add a concurrency guarantee or automatic retry policy from these finite checks. Do not silently rewrite canonical tests.

Step 11 Audit one actual Gemini claim independently

When: 49-55

Where: The actual Gemini service when available and the sanitised prompt file.

Exact file: 03_AI_AUDIT/GEMINI_PROMPT_EN_GB.txt and
03_AI_AUDIT/GEMINI_AUDIT_WORKSHEET_EN_GB.md.

Purpose: Evaluate one claim rather than outsource the assessed implementation.

Action: Supply minimal sanitised context and ask for a bounded review. Do not provide secrets, personal data or the private reference solution. Select one actual claim. Check it against an exact named contract passage or an independent completed observation. Record actual prompt/context, selected claim, check, verdict ACCEPTED/REJECTED/PARTIALLY ACCEPTED/UNKNOWN, correction and a limit of that particular check.

Expected result: A justified rejection can earn full credit. UNKNOWN may be appropriate for a bounded claim; it does not turn unexecuted work into completed execution. An unfinished interaction remains pending.

Evidence: Keep a sanitised actual transcript excerpt or faithful bounded record and the independently sourced check. Distinguish the Gemini statement from your own explanation.

Likely error: Gemini is unavailable, its claim is broad or the check merely asks Gemini again.

Recovery: Use 03_AI_AUDIT/FLAWED_PATCH_OR_CLAIM.txt for clearly labelled synthetic practice. Select a smaller falsifiable claim and a source or observation that does not rely on Gemini's agreement. Keep the actual standard interaction pending when it did not happen.

STOP: Stop at minute 55. Never select ACTUAL_RECORDED for placeholders or synthetic material. A pathway dropdown cannot authorise substitution; preserve any actual teacher decision with its scope and reference.

Step 12 Start the required ADR without a second programming obligation

When: 55-57; finish later

Where: The ADR brief, P03 contract and the ADR fields in the same evidence form.

Exact file: ADR_BRIEF.md, contracts/P03.md and the eight adr_... fields.

Purpose: Make an argued API and transaction decision using traceable source evidence.

Action: Choose a sanitised semester resource or state a provisional scenario. Cover context/invariant, resource identity/representation, at least two alternatives, decision, transaction scope, repeat table, evidence and consequences. Use the eight blank repeat-table rows in the ADR brief. Reading P03 is SOURCE_REASONING. Full P03 and P01 code is optional and is not required for the standard maximum mark. Aim for 300-500 words plus a small table as a teaching target, not a canonical word-count condition.

Identity and repeated effects for the ADR

SCHEMATIC — NOT A CAPTURE OF YOUR CURRENT INTERFACE

P02 booking POST
Create booking operation
Not automatically idempotent
No automatic retry claim

P03 membership PUT / DELETE
Identity: sessionId + attendeeId
Repeated target-state effects
Status category can differ

ADR: eight blank repeat-table rows, source reasoning allowed
Compare alternatives, transaction owner, consequences and review trigger.

V06 What matters: P02 and P03 are separate supplied APIs. What to ignore: no completed ADR answer is shown. A lost response is hypothetical unless actually observed.

Expected result: Your ADR distinguishes P02 booking POST from P03 registration PUT/DELETE. Identical repeated effects need not have identical status codes. A lost response discussed hypothetically is a thought experiment, not measured packet loss.

Evidence: Cite exact supplied paths/sections for expected effects. Record any genuine request separately. Include the ADR in the same final S07 PDF.

Likely error: You treat P03 as an additional mandatory project or copy a completed decision without linking alternatives to your scenario.

Recovery: Return to the eight-part brief. Compare realistic alternatives and explain the chosen resource identity and transaction owner. Keep unresolved assumptions and a review trigger.

STOP: Do not require optional implementation to complete the ADR. Do not invent a GET route for P02 Location or automatically retry the non-idempotent P02 POST.

Step 13 Save close owned resources and STOP at content minute 60

When: 57-60

Where: The evidence form and the original terminal if you started a server.

Exact file: Your form JSON backup and observer/launcher closure records.

Purpose: Preserve the real state of work without inventing completion.

Action: Export the form JSON draft and verify that the downloaded file exists. If you started serve, press Ctrl+C in its original terminal and retain its closure report. STOPPED is meaningful only after owned HTTP and database cleanup is confirmed. A forced kill or timeout is not confirmed cleanup. Write the exit ticket: "Name one observed result, the mechanism it supports and one claim it cannot establish." Identify the smallest pending check.

Expected result: Your editable draft is saved, incomplete work is labelled and the owned server's closure status is known or explicitly uncertain.

Evidence: Record draft filename, pending reason, exit ticket and closure status. A checklist checkmark is progress, not experimental evidence.

Likely error: Browser storage is blocked, export exceeds the stated limit or the server does not close.

Recovery: Keep the form open and preserve the visible text. Use its documented backup/copy route; an export failure must not erase the work. Do not claim a successful save from a button click alone. For uncertain closure, record the output and ask the teacher before further execution.

STOP: At minute 60 stop new content work. Do not kill unrelated processes, claim confirmed cleanup after forced termination or fill observations retrospectively.

Step 14 Complete the standard evidence and inspect one final PDF

When: After the meeting; before the stated deadline

Where: Your reviewed draft and the form's final completeness view.

Exact file: 01_WORKSHEET/index.html or its DOCX alternative, then your one final PDF.

Purpose: Deliver an honest, readable assessment record with all required sections.

Action: Finish pending genuine P02/native and local HTTP witnesses, actual Gemini audit and ADR when the required environment is available and the work is authorised. Review all imported values. The form has seven sections and 65 fields; its 26 core fields support the meeting and its 39 final fields support completion. Use the form's print/PDF route or the editable DOCX route. Inspect every page in the saved PDF, including long observations and the ADR table. Name it TW2026_S07_GROUP_Surname_Firstname.pdf with your actual group/name tokens.

Expected result: One readable PDF contains all seven required sections with truthful evidence classes, limitations and declaration. Exported JSON, DOCX and source ZIP stay private editable materials, not additional final uploads.

Evidence: Record the actual filename, page/readability review, privacy review and any remaining incompleteness. The checker reports structure, not authenticity.

Likely error: Text is cut off, the print preview includes controls or a declaration survived an import without review.

Recovery: Return to the draft, fix layout and print again. The import pathway resets the declaration and actual Gemini state. Review rather than copy an old truth claim. Native browser and Word print acceptance must be checked on the actual surface you use.

STOP: Do not submit a cut-off PDF or claim standard completion with missing genuine/actual evidence. A separate teacher alternative decision, if any, must identify its exact authorised scope.

Step 15 Use the one S07 Assignment and verify final submission

When: When the owner has configured Moodle

Where: The configured S07 Assignment on the actual learning platform.

Exact file: The one reviewed TW2026_S07_GROUP_Surname_Firstname.pdf.

Purpose: Distinguish upload, saved draft and final submitted status.

Action: Follow the actual configured Assignment instructions and deadline. Upload the single PDF. Inspect the filename and the platform's final submission state after any required submit action. The Moodle drawing here is a proposal, not a current interface capture; buttons and confirmation wording can vary. If the Assignment is absent, record the block and keep the reviewed PDF.

One final PDF and final submission state

SCHEMATIC — NOT A CAPTURE OF YOUR CURRENT INTERFACE

EDITABLE DRAFTS

Form JSON / optional DOCX
Private working materials

ONE FINAL S07 PDF

TW2026_S07_GROUP_
Surname_Firstname.pdf

Seven sections reviewed · privacy checked · truthful declaration
Moodle proposed state: saved draft ≠ final submitted
Use actual configured instructions and later stated deadline.

V07 What matters: inspect every PDF page and the actual final submission state. **What to ignore:** the diagram is not live Moodle and names no current buttons or deadline.

Expected result: The platform shows your correct PDF and a final submitted state, or you explicitly retain a pending platform block. A saved draft alone is not final submission.

Evidence: Retain a sanitised record of the actual final state and time where permitted. Exclude account names, other students and private details from shared captures.

Likely error: You see only draft/saved, upload a JSON/ZIP or look for a separate C07 Assignment.

Recovery: Use the actual finalise/submit route if configured. Replace an incorrect file before finalising. There is one S07 Assignment and no second C07 submission for this activity.

STOP: Do not invent a deadline, platform configuration or successful final state. Contact the teacher if the actual interface prevents completion.

Eight planned experiments a blank evidence plan not completed results

Every row below is a deterministic teaching plan derived from the supplied code/contracts. **Production status: NOT_EXECUTED for the canonical stack, HTTP, Gemini and native UI.** Fill your own actual records. IDs retain the contract's E01-E06 families. E03_CALLBACK and E03_AUDIT are separate E03 failure witnesses; G01_GEMINI is the bounded AI activity. In observer output E02_SERVICE and E02_HTTP are separate success witnesses, E04 has named case suffixes and the E05 five-operation witness is embedded in E02_SERVICE. These identifiers never imply that the scenario ran.

E01 Unsafe partial write comparison

Input/fixture: Fresh event 1, capacity/availability 5, zero Booking/Audit; attendee 42, seats 2; reject after Booking creation.

Prior prediction: Predict the three state facts. Source expectation is unsafe 3/1/0.

Record and limit: Record the actual unsafe state or NOT_EXECUTED. This is a sequential contrast, not a concurrency experiment.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

E02 Managed success and local HTTP

Input/fixture: A separate fresh fixture with the same named two-seat request; supplied application POST.

Prior prediction: Predict managed state and detached response; expected 3/1/1 and HTTP 201.

Record and limit: Keep ORM state, outer settlement and actual HTTP as separate witnesses. No GET booking-member route is supplied.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

E03 CALLBACK Callback rejection then same instance recovery

Input/fixture: A separate fresh fixture; awaited afterBookingCreated rejects after Booking creation.

Prior prediction: Predict equality of all before/after state facts and later independent success.

Record and limit: Preserve error identity/origin. Recovery must use the same instance that experienced the failure.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

E03 AUDIT Audit create rejection then same instance recovery

Input/fixture: A separate fresh fixture; reject inside BookingAudit.create.

Prior prediction: Predict equality of before/after and later success.

Record and limit: Do not confuse this with throwing before the operation. A pre-insert failure does not establish an escaped audit row.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

E04 Finite edge rejection and precedence cases

Input/fixture: Separate fixtures for seats 0/non-integer, missing event, exact capacity 5, request 6 and duplicate.

Prior prediction: Predict rejection/unchanged state where required. Follow missing → capacity → duplicate precedence.

Record and limit: Name each input and observed state. A duplicate with insufficient stock can report sold_out. No universal ID-validation claim.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

E05 Five operation option and settlement interpretation

Input/fixture: A managed success trace for all five ORM operations and callback; unchanged canonical spy.

Prior prediction: Predict one shared non-null transaction and awaited sequence.

Record and limit: Read the separate observer trace. Four-method spy alone is insufficient. Model identity evidence is not native isolation proof.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

G01 GEMINI Actual bounded Gemini audit

Input/fixture: One actual sanitised prompt and selected claim; independent source passage or genuine check.

Prior prediction: Predict what smallest evidence would distinguish the claim.

Record and limit: Record actual input/claim/check/verdict/correction/claim-specific limit. Offline practice remains synthetic.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

E06 Required P03 informed ADR

Input/fixture: Sanitised actual or explicitly provisional semester resource; eight P03 repeat-table rows.

Prior prediction: Predict repeated target-state effects, not identical status codes.

Record and limit: SOURCE_REASONING is allowed for ADR. Full P03 implementation is optional. Lost response is hypothetical unless measured.

Your status: _____ · **class:** _____ · **command/source:** _____ · **before/after:** _____ ·
remaining check: _____

Tools only when needed

Tool	Use	If unavailable
File manager and archive tool	Extract once, inspect filenames and preserve ZIP/sidecar	Keep the block and request the correct kit
VS Code or suitable text editor	Edit the one P02 file and view the breadcrumb	Use an authorised suitable editor; retain the exact file boundary
Node/npm reference runtime	New CLI checks and later guarded genuine execution	Record actual versions and pending readiness; do not silently install
Project-local Express/Sequelize/sqlite 3	Genuine supplied P02 application/tests	SOURCE_REASONING or SYNTHETIC_PRACTICE remains separately labelled
Browser	Offline guide/form and later actual local HTTP evidence	Use the Markdown/DOCX fallback; no native qualification is presumed
Gemini	One bounded actual review	Synthetic practice only; actual standard activity stays pending
Word or suitable DOCX editor	Optional editable evidence route	HTML form is the other route; inspect the actual exported PDF
SQLite CLI/DB Browser	Optional inspection	Not a required extra tool or upload
Configured Moodle	One final PDF submission	Keep the reviewed PDF and exact platform block

Browser and operating system routes

Choose Windows, macOS or Linux and Chrome/Edge/Firefox in the interactive guide to keep a local progress label. This choice is not an observed version and does not grant permission. Record the actual version in the evidence form. Usual browser developer-tools routes are F12 or the browser menu's developer tools entry on Windows/Linux, and the browser's Tools/Develop menu or its usual shortcut on macOS. Interface wording, shortcuts and availability must be checked on your actual browser. In Network, a page loaded from `file:` is not application HTTP evidence. Use the server's actual printed `http://127.0.0.1:PORT` origin when the genuine application is running. Do not publish private screenshots.

Keyboard users can tab to links, selectors and progress controls. The guide preserves visible focus, supports reduced motion and has a high-contrast control. Copy buttons use the clipboard only when you request it; if blocked, select the visible command and copy manually. Progress saving is optional and local. Storage may be blocked or separated by file origin. Export a guide-progress backup when useful; it contains checklist state only and is separate from the assessed form JSON.

Recovery wizard choose the smallest truthful next step

Symptom	Check first	Safe recovery	Stop condition
Missing guide assets	ZIP preview versus real extracted directory	Open the extracted 00_START_HERE/index.html; text fallback is GUIDE.md	Corrupt/mismatched kit
Cannot find tools/cli.mjs	Terminal working directory	Navigate to student root	Root identity unknown
Dependency/runtime/native block	Exact preflight/native output	Preserve it; source reasoning while readiness is arranged	No genuine run claim
Unexpected objective failure	Actual command, fixture and permitted diff	Read contract and smallest reported mismatch	Do not alter protected tests
Transaction trace incomplete	All five operations and non-null identity	Use separate observer and preserve trace provenance	No native/concurrency extrapolation
Local page/404 confusion	Actual READY origin, method and supplied route	Use printed URL; do not follow Location as an invented GET	No foreign-process kill
Form backup/print failure	Visible failure and current text	Retain text, retry documented export/print and inspect every page	No false “saved” claim
Gemini unavailable	Actual service block	Label offline claim synthetic; real activity pending	No selector-authorised substitution
Moodle draft only	Actual configured final submission route	Review one PDF then finalise if required	No invented submitted state

Glossary

Invariant: facts that must agree. **Managed transaction:** the ORM owns commit or rollback around one awaited callback. **Rollback:** the database changes in that boundary are undone after rejection. **Fixture:** the named starting data for a check. **Detached result:** a plain representation separated from a live ORM instance. **Outer settlement:** the caller’s promise resolves or rejects after the managed transaction has finished its own process. **Idempotent effect:** repeating the intended action reaches the same target state; response status can differ. **Error provenance:** where an error actually arose, beyond its class label. **Evidence class:** the scope of the observation or reasoning. **ADR:** a short record of a design choice, alternatives, evidence and consequences.

Completion and marking

The adopted rubric has six groups and 10 points: technical result 3, reproducible evidence 2, mechanism 2, critical Gemini audit 1.5, limitations/reflection 1 and completeness/format 0.5.

Mechanism reserves one point for P02 explanation and one for the required ADR. Prior prediction chronology is scored under reproducibility. Evidence of a result and explanation of that result are distinct; the same technical result is not counted twice. See `00_START_HERE/ASSESSMENT_CRITERIA_EN_GB.md` for the authoritative allocation.

At the end you should be able to say: one result I actually observed, the mechanism it supports, one claim it cannot establish and the smallest check still pending. Keep the genuine/source/model distinction even when a result matches the expectation.